

Secure Coding Mastery Course

Stack used for examples: Python + FastAPI + PostgreSQL (SQLAlchemy / asyncpg / Pydantic)

This is a **living curriculum**. It's too large for one document to do justice to, so this first file gives you:

1. The full **roadmap** (every module we will cover, in order)
2. A table comparing **every OWASP Top 10 edition** (2004 → 2021) so you see how the threat landscape evolved over ~20 years, not just "this year's list"
3. The **extended vulnerability catalog** — everything beyond the Top 10
4. A short note on where **AI vibe-coding security** fits in
5. **Module 1, fully worked:** SQL Injection — theory → attack → exploit → fix → tools, with 5 progressively harder FastAPI + Postgres code versions

Each future message will deliver one or two modules at this same depth. We go in the order listed in the roadmap unless you tell me to jump around.

1. Roadmap (the order we'll go in)

Part A — Injection & Input Trust

1. SQL Injection (*this document*)
2. NoSQL / Command / LDAP / XPath / Template Injection
3. Cross-Site Scripting (XSS) — reflected, stored, DOM-based
4. Server-Side Request Forgery (SSRF)
5. XML External Entity (XXE) & unsafe deserialization

Part B — AuthN / AuthZ 6. Broken Authentication (passwords, sessions, brute force, credential stuffing) 7. Broken Access Control (IDOR, privilege escalation, mass assignment) 8. JWT & OAuth2/OIDC pitfalls 9. Multi-factor auth & session management done right

Part C — Data & Crypto 10. Sensitive Data Exposure / Cryptographic Failures (hashing vs encryption, TLS, secrets management) 11. Insecure Direct Object References & data-layer authorization in FastAPI+Postgres

Part D — App & Infra Configuration 12. Security Misconfiguration (headers, CORS, debug mode, default creds) 13. Using Components with Known Vulnerabilities (SCA, dependency pinning, SBOM) 14. Security Logging & Monitoring Failures 15. CSRF (and why it changed shape over the years) 16. Clickjacking & UI redress 17. Rate limiting, DoS, resource exhaustion (ReDoS, zip bombs, unbounded queries) 18. File upload vulnerabilities (path traversal, RCE via upload, content-type spoofing) 19. Business logic vulnerabilities (race conditions, TOCTOU, workflow bypass) 20. Supply chain attacks (typosquatting,

dependency confusion, malicious packages) 21. SSTI (Server-Side Template Injection) deep dive 22. Insecure design (threat modeling basics, STRIDE)

Part E — Modern / AI-era topics 23. Secure coding with AI pair-programmers / "vibe coding" — prompt-injection-in-code, hallucinated packages, blind trust in generated auth/crypto code 24. Secrets in prompts/LLM logs, AI-assisted code review, SAST/DAST/dependency scanning in CI

Part F — Wrap-up 25. Secure SDLC end-to-end for a FastAPI+Postgres service (checklist you keep)

2. OWASP Top 10 — every edition side by side

OWASP has published a Top 10 six times: **2003 (first draft), 2004, 2007, 2010, 2013, 2017, and 2021** (the 2021 edition is still current as of 2026 — a new one has not been released yet). Ranks shift because the *industry* changes (frameworks auto-escape more, ORMs became standard, cloud misconfig became common, APIs replaced server-rendered apps).

Rank	2013	2017	2021 (current)
1	Injection	Injection	Broken Access Control
2	Broken Authentication	Broken Authentication	Cryptographic Failures
3	XSS	Sensitive Data Exposure	Injection (XSS merged in here)
4	Insecure Direct Object References	XXE	Insecure Design (<i>new category</i>)
5	Security Misconfiguration	Broken Access Control	Security Misconfiguration
6	Sensitive Data Exposure	Security Misconfiguration	Vulnerable & Outdated Components
7	Missing Function Level Access Control	XSS	Identification & Authentication Failures
8	CSRF	Insecure Deserialization	Software & Data Integrity Failures (<i>new — supply chain</i>)
9	Using Components with Known Vulnerabilities	Using Components w/ Known Vulns	Security Logging & Monitoring Failures
10	Unvalidated Redirects & Forwards	Insufficient Logging & Monitoring	Server-Side Request Forgery (SSRF) (<i>new</i>)

What changed and why it matters:

- **XSS dropped off as its own #1-3 slot** — not because it vanished, but because modern frameworks (React, Jinja2 autoescaping, FastAPI+Pydantic) auto-escape output by default. It's folded into "Injection" in 2021.
- **CSRF disappeared from the 2021 list entirely** — most frameworks now ship CSRF tokens or SameSite cookies by default. It's still real, especially in APIs that skip cookies-based auth carelessly, so we'll still cover it.
- **Broken Access Control jumped to #1** in 2021 — as apps became API-driven (like FastAPI services), authorization bugs (IDOR, missing ownership checks) became the most commonly found flaw in real audits.
- **Insecure Design** and **Software/Data Integrity Failures** are brand new categories — reflecting that a well-implemented but badly-designed system is still exploitable, and that supply-chain attacks (SolarWinds-style, malicious npm/pip packages) became mainstream threats.
- The pre-2013 editions (2004, 2007, 2010) mostly established the vocabulary: Injection and XSS have been #1-2 in *every single edition since 2004* — 20+ years running. That consistency is itself a lesson: the fundamentals rarely change, only their delivery mechanism does.

We will cover **all ten from 2021**, plus everything **dropped or renamed** from earlier editions (CSRF, Unvalidated Redirects, Insecure Deserialization, XXE — all still real, just reclassified), so nothing falls through the cracks.

3. Extended vulnerability catalog (beyond any Top 10 list)

These matter in real-world Python/FastAPI/Postgres systems and are covered as their own modules above:

- Race conditions / TOCTOU (e.g., double-spend on a wallet balance)
 - ReDoS (catastrophic regex backtracking)
 - Mass assignment (Pydantic model over-binding — very FastAPI-specific)
 - Prototype pollution (JS-side, relevant if you pair FastAPI with a JS frontend)
 - Timing attacks on comparisons (password/token comparison without `hmac.compare_digest`)
 - Insecure randomness (`random` vs `secrets`)
 - Host header injection / cache poisoning
 - Open redirect
 - GraphQL-specific issues (if you ever swap REST for GraphQL — introspection, query depth)
 - Container/deployment issues (running as root, secrets baked into images)
 - Dependency confusion / typosquatting attacks on PyPI
-

4. AI vibe-coding security — the short version (full module later)

When you or an AI assistant generates code quickly:

- **Never trust generated SQL, auth, or crypto code blindly** — these are exactly the places where "looks right" and "is right" diverge the most.
 - AI models sometimes **hallucinate package names** — always verify a package exists and is the real one on PyPI before installing (this is a live supply-chain attack vector called *slopsquatting*).
 - Treat AI-written code the same as a junior dev's PR: it still needs the same input validation, parameterization, and access-control review this course teaches.
 - We'll build a checklist for this once you've seen the underlying vulnerabilities — you can't review AI code for injection risk if you don't yet know what injection looks like. That's why this topic comes near the end, not the beginning.
-

MODULE 1 — SQL Injection (SQLi)

1.1 What it is (theory, beginner-level)

Your app builds a SQL query using data from the user, then sends that string to the database. If you build the query by **gluing strings together**, a user can type input that changes the *structure* of the query instead of just its *data*.

Think of it like a mail-merge letter: `"Dear {name}, your balance is {amount}"`. If someone's "name" is allowed to contain `{amount}: $999999`, and the merge tool blindly substitutes text, they've now rewritten part of the template itself — not just filled in a blank. SQL injection is the same idea: the user input escapes the "data" role and becomes "code".

1.2 The attack — a concrete walkthrough

Imagine a login query built like this (do not use this — shown to explain the attack):

```
python

query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{pa
```

If the app expects `username = alice` and `password = hunter2`, the query becomes:

```
sql

SELECT * FROM users WHERE username = 'alice' AND password = 'hunter2'
```

Now imagine the attacker enters this as the **username**, with password left blank:

```
' OR '1'='1' --
```

The final query becomes:

```
sql
SELECT * FROM users WHERE username = '' OR '1'='1' --' AND password = ''
```

Breaking that down:

- `' OR '1'='1'` — closes the string early, then adds a condition that is **always true**
- `--` — starts a SQL comment, so everything after it (the password check) is ignored

Result: the query now matches **every row in the table**, and most `SELECT * ... LIMIT 1` style code returns the *first user in the database* — often an admin — logging the attacker in without ever knowing a password. This exact technique is decades old and is still found in real audits today.

Other things an attacker can do once injection exists:

- `UNION SELECT` to pull data out of *other tables* (e.g., dump every password hash)
- Boolean/time-based **blind injection** — asking yes/no questions like "does the DB version start with 14?" by observing `AND 1=1` (true) vs `AND 1=2` (false) response differences, or by injecting `pg_sleep(5)` and measuring response time — this lets an attacker exfiltrate an entire database *without any error messages*, one bit at a time
- Stacked queries: `; DROP TABLE users; --` (Postgres allows multiple statements in some execution paths — `psycopg2/asyncpg` block this by default when using parameterization, which is one more reason parameterization matters)

1.3 The fix — theory

Never build a query by string concatenation/f-strings with user input. Instead, send the query text and the user data **separately** to the database driver. The driver sends them to Postgres as distinct messages — Postgres treats the data purely as data, no matter what characters it contains, because it was never re-parsed as SQL text.

This is called a **parameterized query** (or "prepared statement"). It's not a filter, not a blocklist of dangerous characters, not "escaping quotes" — it's a structural fix that makes the attack impossible by design, not just unlikely.

1.4 Libraries/tools for Python + FastAPI + Postgres

Purpose	Tool
Low-level Postgres driver (sync)	<code>psycopg2</code> / <code>psycopg</code> (v3) — supports parameterized queries natively

Purpose	Tool
Low-level Postgres driver (async)	<code>asyncpg</code>
ORM (recommended for most apps)	<code>SQLAlchemy</code> (2.0 style, async support)
Request/response validation	<code>Pydantic</code> (built into FastAPI) — stops malformed types before they reach your query layer
Static analysis to catch raw SQL concatenation	<code>bandit</code> (<code>B608: hardcoded_sql_expressions</code>)
Dependency/vuln scanning	<code>pip-audit</code> , <code>safety</code>
Runtime protection (defense in depth)	least-privilege DB roles, WAF (e.g., Cloudflare/AWS WAF) — never a substitute for parameterization

Install for the examples below:

```
bash
pip install fastapi uvicorn "sqlalchemy[asyncio]" asyncpg psycopg2-binary pydan
```

1.5 Progressive code examples

We'll build the *same* "get user by username" endpoint five times, each version fixing the previous version's flaw, so you can see exactly what "more secure" looks like in practice.

Level 1 — Vulnerable (never deploy this — for learning only)

```
python
```

```
# level1_vulnerable.py
from fastapi import FastAPI
import psycopg2

app = FastAPI()

def get_db():
    return psycopg2.connect(
        host="localhost", dbname="appdb", user="appuser", password="apppass"
    )

@app.get("/users/search")
def search_user(username: str):
    conn = get_db()
    cur = conn.cursor()
    # VULNERABLE: user input is glued directly into SQL text
    query = f"SELECT id, username, email FROM users WHERE username = '{username}'"
    cur.execute(query)
    row = cur.fetchone()
    conn.close()
    return {"id": row[0], "username": row[1], "email": row[2]} if row else {}
```

Exploit against this exact endpoint:

```
GET /users/search?username=x' UNION SELECT id, username, password_hash FROM
users --
```

This dumps every user's password hash through a field that was only ever meant to show an email address — because the attacker controls the *shape* of the query, not just a value in it.

Level 2 — A common half-fix that's still broken

A beginner sometimes "fixes" this by trying to sanitize the string manually:

```
python
```

```
@app.get("/users/search")
def search_user(username: str):
    # STILL VULNERABLE: naive blocklist, easily bypassed
    safe_username = username.replace("'", "")
    query = f"SELECT id, username, email FROM users WHERE username = '{safe_username}'"
    cur.execute(query)
    ...
```

Why this fails: you're trying to enumerate every dangerous character/pattern yourself (quotes, comments, backslashes, encoded variants, Unicode lookalikes...). Blocklists lose this race almost every time — real Postgres query parsers understand far more syntax than a simple `.replace()` call can anticipate. **Never hand-roll sanitization for SQL.**

Level 3 — Real fix: parameterized query with the raw driver

```
python
# level3_parameterized.py
from fastapi import FastAPI
import psycopg2

app = FastAPI()

def get_db():
    return psycopg2.connect(
        host="localhost", dbname="appdb", user="appuser", password="apppass"
    )

@app.get("/users/search")
def search_user(username: str):
    conn = get_db()
    cur = conn.cursor()
    # SAFE: %s is a placeholder; psycopg2 sends the value separately from the SQL
    cur.execute(
        "SELECT id, username, email FROM users WHERE username = %s",
        (username,)
    )
    row = cur.fetchone()
    conn.close()
    return {"id": row[0], "username": row[1], "email": row[2]} if row else {}
```

Now `username = 'x' OR '1'='1'` is treated as **one literal string value** to compare against the `username` column — it can never change the query's structure, no matter what it contains.

Level 4 — Idiomatic FastAPI: async SQLAlchemy ORM + Pydantic validation

```
python
```

```

# level4_sqlalchemy.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy.orm import sessionmaker, declarative_base
from sqlalchemy import Column, Integer, String, select

DATABASE_URL = "postgresql+asyncpg://appuser:apppass@localhost/appdb"
engine = create_async_engine(DATABASE_URL)
SessionLocal = sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)
Base = declarative_base()

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    username = Column(String, unique=True, nullable=False)
    email = Column(String, nullable=False)

class UserOut(BaseModel):
    id: int
    username: str
    email: str

app = FastAPI()

@app.get("/users/search", response_model=UserOut)
async def search_user(
    # Pydantic/FastAPI validates type + length BEFORE this ever reaches the DB
    username: str = Field(min_length=1, max_length=64)
):
    async with SessionLocal() as session:
        # SAFE: SQLAlchemy always parameterizes values passed to .where()
        result = await session.execute(select(User).where(User.username == username))
        user = result.scalar_one_or_none()
        if not user:
            raise HTTPException(status_code=404, detail="User not found")
        return user

```

New things this version adds:

- The ORM **always** parameterizes comparisons like `User.username == username` — you'd have to go out of your way (e.g. `session.execute(text(f"..."))`) to reintroduce the bug
- `Field(min_length=1, max_length=64)` rejects garbage input (e.g., a 50,000-character string used for a denial-of-service or buffer-stress attempt) before it's even processed

- `response_model=UserOut` ensures you only ever return the fields you intend — you can't accidentally leak `password_hash` even if it's on the `User` model, because Pydantic filters the output shape
-

Level 5 — Defense in depth: least privilege, logging, and safe raw SQL escape hatch

Sometimes you legitimately need raw SQL (complex reporting query, DB-specific feature). Here's how to do that *safely*, plus two more layers of protection a real production service should have:

```
python
```

```

# level5_defense_in_depth.py
import logging
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy.orm import sessionmaker
from sqlalchemy import text

# --- 1. Least-privilege DB user ---
# `readonly_appuser` should be a Postgres role created like:
# CREATE ROLE readonly_appuser LOGIN PASSWORD '...';
# GRANT SELECT ON users TO readonly_appuser; -- no INSERT/UPDATE/DELETE/DRO
# Even if injection somehow occurred, this role physically cannot modify or dro
DATABASE_URL = "postgresql+asyncpg://readonly_appuser:apppass@localhost/appdb"
engine = create_async_engine(DATABASE_URL)
SessionLocal = sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)

logger = logging.getLogger("security")

class UserOut(BaseModel):
    id: int
    username: str
    email: str

app = FastAPI()

@app.get("/users/search", response_model=UserOut)
async def search_user(username: str = Field(min_length=1, max_length=64)):
    async with SessionLocal() as session:
        try:
            # --- 2. Raw SQL done SAFELY: named bind parameters via text() ---
            # The value is still sent separately from the SQL text – this is NO
            # an f-string. This is the correct way to use raw SQL when you need
            result = await session.execute(
                text("SELECT id, username, email FROM users WHERE username = :u
                {"uname": username},
            )
            user = result.mappings().first()
        except Exception:
            # --- 3. Fail closed + log for detection (OWASP #9: Logging & Monit
            logger.warning("DB query failed for username lookup", exc_info=True)
            raise HTTPException(status_code=500, detail="Internal error")

    if not user:

```

```
raise HTTPException(status_code=404, detail="User not found")
return UserOut(**user)
```

What's layered in here, and why each layer matters even though Level 4 already "fixed" SQLi:

- **Least privilege:** if a *different* bug (not even this endpoint) ever let an attacker run arbitrary SQL, a read-only DB role limits the blast radius to "can read the users table" — not "can drop the whole database." This is the single highest-leverage mitigation for the *impact* of a successful attack, independent of preventing the attack itself. Tool: managed via plain Postgres `GRANT`/`REVOKE`, or provisioned via Terraform/IaC in production.
- **Safe raw SQL:** `text(":uname")` + a params dict is still fully parameterized — this is the pattern to reach for when the ORM's query builder can't express something you need, instead of falling back to an f-string.
- **Logging on failure:** OWASP's #9 (2021) is "Security Logging & Monitoring Failures" — many real breaches went undetected for months because failed/suspicious queries were never logged anywhere a human would see them.

1.6 Testing this yourself (safely, locally)

```
bash

# Start a local disposable Postgres for practice
docker run --name secdemo -e POSTGRES_PASSWORD=apppass -e POSTGRES_USER=appuser
-e POSTGRES_DB=appdb -p 5432:5432 -d postgres:16

# Run bandit against level1 to see it flag the vulnerable pattern
bandit level1_vulnerable.py
```

`bandit` should flag `B608` on the f-string query in Level 1, and report clean on Levels 3–5. Try the actual `' OR '1'='1' --` payload against Level 1 vs Level 3 in a browser or `curl` and watch the difference in behavior — seeing it fail against the fixed version is the best way to cement this.

What's next

That's the full depth for Module 1. Say "**next**" (or name a topic from the roadmap) and I'll deliver the next module at this same level — theory, live attack walkthrough, the fix, tooling, and progressively harder FastAPI+Postgres code.

